

Parte 2. Comprensión del flujo colaborativo

1. Flujo colaborativo

Planteamiento: Supón que quieres colaborar con el repositorio de otro desarrollador. Ordena y explica los siguientes elementos: Push, Fork, Pull Request, Clone, Merge, Commit, Review, Branch y Modificar archivos. Agrega cualquier operación que consideres necesaria.

Respuesta: El orden correcto es:

1. **Fork** - Crear, dentro de mi propia cuenta de GitHub, una copia del repositorio original.
2. **Clone** - Descargar esa copia (mi fork) a mi computadora para poder trabajar en ella localmente.
3. **Branch** - Crear una rama nueva dentro de mi repositorio local, dedicada al cambio que voy a realizar.
4. **Modificar archivos** - Realizar los cambios necesarios en el código o documentación dentro de esa rama.
5. **Commit** - Registrar esos cambios como una instantánea en el historial local del repositorio, con un mensaje descriptivo.
6. **Push** - Subir la rama y sus commits desde mi repositorio local hacia mi fork en GitHub.
7. **Pull Request** - Solicitar formalmente, desde GitHub, que los cambios de mi rama sean incorporados al repositorio original.
8. **Review** - El propietario o los mantenedores del repositorio original revisan el código propuesto, dejan comentarios y deciden si aprobarlo o solicitar cambios.
9. **Merge** - Una vez aprobado el Pull Request, los cambios se integran (fusionan) a la rama principal del repositorio original.

Operación adicional que agregaría: **Sync Fork / git pull**, ya utilizada al inicio del proceso (o repetida antes de crear una nueva rama), para asegurarme de que mi fork y mi copia local están actualizados con respecto al repositorio original antes de empezar a trabajar, evitando conflictos.

Explicación: Este flujo refleja el modelo de colaboración típico de proyectos de código abierto (o de repositorios donde no tengo permisos de escritura directos). Primero se obtiene una copia propia del proyecto (Fork y Clone), luego se aísla el trabajo en una rama para no afectar la rama principal (Branch), se realiza y registra el trabajo (Modificar archivos y Commit), se comparte ese trabajo en GitHub (Push) y finalmente se propone formalmente su incorporación al proyecto original (Pull Request), lo cual habilita un proceso de control de calidad (Review) antes de que el cambio pase a formar parte definitiva del proyecto (Merge).

2. Fork y Clone

Planteamiento: Analiza la afirmación: "Clone crea una copia del proyecto dentro de mi cuenta de GitHub". Indica si es correcta y explica la diferencia entre Fork y Clone.

Respuesta: La afirmación es **incorrecta**.

Explicación: Clone no crea nada dentro de mi cuenta de GitHub; **Clone** es la operación que descarga (copia) un repositorio -ya sea el original, un fork mío, o cualquier otro al que tenga acceso- desde GitHub hacia mi computadora, generando una copia local con la que puedo trabajar y a la que Git puede hacer seguimiento de cambios. Es una operación que ocurre del lado del cliente (mi máquina).

Fork, en cambio, sí crea una copia del repositorio dentro de mi propia cuenta de GitHub. Es una operación que ocurre del lado del servidor (en GitHub), y su propósito es darme un repositorio propio, independiente del original, sobre el cual tengo permisos completos de escritura, para poder proponer cambios a un proyecto en el que no tengo permisos directos.

En resumen: Fork copia el repositorio a mi cuenta de GitHub (en la nube); Clone copia un repositorio (el original o un fork) a mi computadora (en local). Ambas operaciones suelen usarse juntas: primero se hace Fork y después se hace Clone del fork.

3. Pull Request

Planteamiento: Supón que realizaste Fork, Clone, Branch, Modificar, Commit y Push. Responde: ¿los cambios ya forman parte del repositorio original? ¿Qué debe ocurrir para incorporarlos?

Respuesta: No, los cambios **todavía no forman parte del repositorio original**. Después de hacer Push, esos commits únicamente existen en mi repositorio (mi fork) dentro de GitHub, en la rama que creé. El repositorio original no ha sido modificado en absoluto.

Para incorporarlos al repositorio original debe ocurrir lo siguiente: debo **abrir un Pull Request** desde mi rama (en mi fork) hacia la rama correspondiente del repositorio original. El propietario o los mantenedores del proyecto original revisarán el Pull Request (Review) y, si lo aprueban, realizarán el **Merge**, momento en el cual mis cambios sí pasan a formar parte oficial del repositorio original.

Explicación: Push y Pull Request cumplen funciones distintas: Push solamente sincroniza mi repositorio local con mi repositorio remoto (mi fork); es una operación entre "mis" repositorios. Pull Request es la solicitud formal de incorporar cambios entre dos repositorios distintos (el mío y el original), y requiere de una acción explícita de aprobación (Merge) por parte de quien administra el repositorio original.

4. Request Changes

Planteamiento: El propietario revisa tu Pull Request y selecciona Request Changes. Explica qué debes hacer, si necesitas crear otro Pull Request y qué ocurre cuando realizas nuevamente push.

Respuesta: Cuando el propietario selecciona **Request Changes**, debo corregir mi código en mi

repositorio local: modifíco los archivos indicados según los comentarios de la revisión, y registro esas correcciones con uno o varios **commits** nuevos. Después ejecuto **push** hacia la **misma rama** que ya estaba vinculada al Pull Request.

No es necesario crear otro Pull Request. El Pull Request está asociado directamente a una rama específica de mi repositorio (no a un conjunto fijo de commits), así que cualquier commit nuevo que suba a esa misma rama se añade automáticamente al Pull Request ya existente.

Cuando hago push nuevamente, GitHub actualiza el Pull Request de forma automática mostrando los nuevos commits, y normalmente vuelve a marcar la conversación para que el revisor la revise otra vez y decida si aprueba los cambios o solicita más ajustes.

Explicación: Este comportamiento es lo que hace eficiente el flujo de revisión de código: en lugar de reiniciar el proceso con un Pull Request distinto cada vez que hay observaciones, el mismo Pull Request evoluciona con cada push a la rama, conservando el historial completo de la conversación y de los cambios realizados durante la revisión.

5. Merge y repositorio local

Planteamiento: Un Pull Request fue aceptado y se realizó Merge en GitHub. Sin embargo, el repositorio local del propietario no contiene los cambios. Explica por qué sucede y qué operación debe realizarse.

Respuesta: Esto sucede porque el **Merge** realizado en GitHub es una operación que ocurre únicamente en el **repositorio remoto** (en los servidores de GitHub). Git no sincroniza automáticamente los repositorios locales de las personas cuando algo cambia en el remoto; la sincronización siempre requiere una acción explícita.

Para que el propietario tenga esos cambios en su computadora, debe ejecutar ``git pull`` (o bien ``git fetch`` seguido de ``git merge``) sobre su repositorio local, estando posicionado en la rama correspondiente (por ejemplo, ``main``), de modo que descargue del remoto los nuevos commits generados por el Merge y actualice su copia local.

Explicación: Es un error común asumir que "aceptar" un Pull Request actualiza automáticamente todas las copias del proyecto. En realidad, cada copia local de un repositorio es independiente hasta que alguien decide explícitamente sincronizarla (push para subir cambios, pull para bajarlos). Por eso, incluso el propio dueño del repositorio debe hacer pull después de un merge para ver esos cambios reflejados en su máquina.

6. Sync Fork

Planteamiento: Tu Fork fue creado varios días atrás y el repositorio original recibió nuevos commits. Explica qué herramienta utilizarías, qué repositorio se actualiza y qué diferencia existe entre Sync Fork y git pull.

Respuesta: Utilizaría la opción "**Sync fork**" que ofrece GitHub en la página web de mi fork (o, de forma equivalente desde la terminal, agregando el repositorio original como remoto ``upstream`` y ejecutando ``git fetch upstream`` seguido de ``git merge upstream/main``).

El repositorio que se actualiza con Sync Fork es **mi fork, alojado en GitHub** (el repositorio remoto en la nube que está bajo mi cuenta), no mi repositorio local.

Diferencia entre Sync Fork y git pull:

- **Sync Fork** actualiza mi **fork en GitHub** trayendo los commits nuevos desde el **repositorio original (upstream)**, que es un repositorio distinto al mío. Es una operación entre dos repositorios remotos distintos.

- **git pull** actualiza mi **repositorio local** trayendo los cambios desde un repositorio remoto que ya tengo configurado (normalmente ``origin``, es decir, mi propio fork). No tiene relación directa con el repositorio original a menos que yo haya configurado explícitamente ``upstream`` como remoto adicional.

Explicación: En un flujo de trabajo con Fork, en realidad existen tres copias del proyecto: el repositorio original (upstream), mi fork en GitHub (origin) y mi copia local. Mantenerlas sincronizadas requiere dos pasos distintos: primero traer los cambios del original hacia mi fork (Sync Fork), y después traer los cambios de mi fork hacia mi copia local (git pull). Omitir el primer paso puede hacer que mi fork quede desactualizado y que, al intentar proponer cambios, se generen conflictos con el proyecto original.